

定点乘法器性能优化实验报告

2026 年 4 月 12 日

1 实验目标

1. 理解原始 32 位串行乘法器的工作原理（32 个时钟周期完成运算）。
2. 对乘法器进行最简单有效的性能优化，将时钟周期从 32 周期减少至 16 周期。
3. 采用 2 位并行乘法方式，实现硬件开销小、实现最简单的优化方案。
4. 完成功能仿真与上板验证，确保优化后乘法器结果正确。
5. 对比优化前后性能，完成实验分析与总结。

2 实验原理图

2.1 2 位并行乘法器结构原理图

本实验采用 2 位并行乘法优化方案，每次同时读取乘数低 2 位，根据不同组合生成 0 倍、1 倍、2 倍、3 倍被乘数，作为部分积参与累加，从而将时钟周期减少一半。

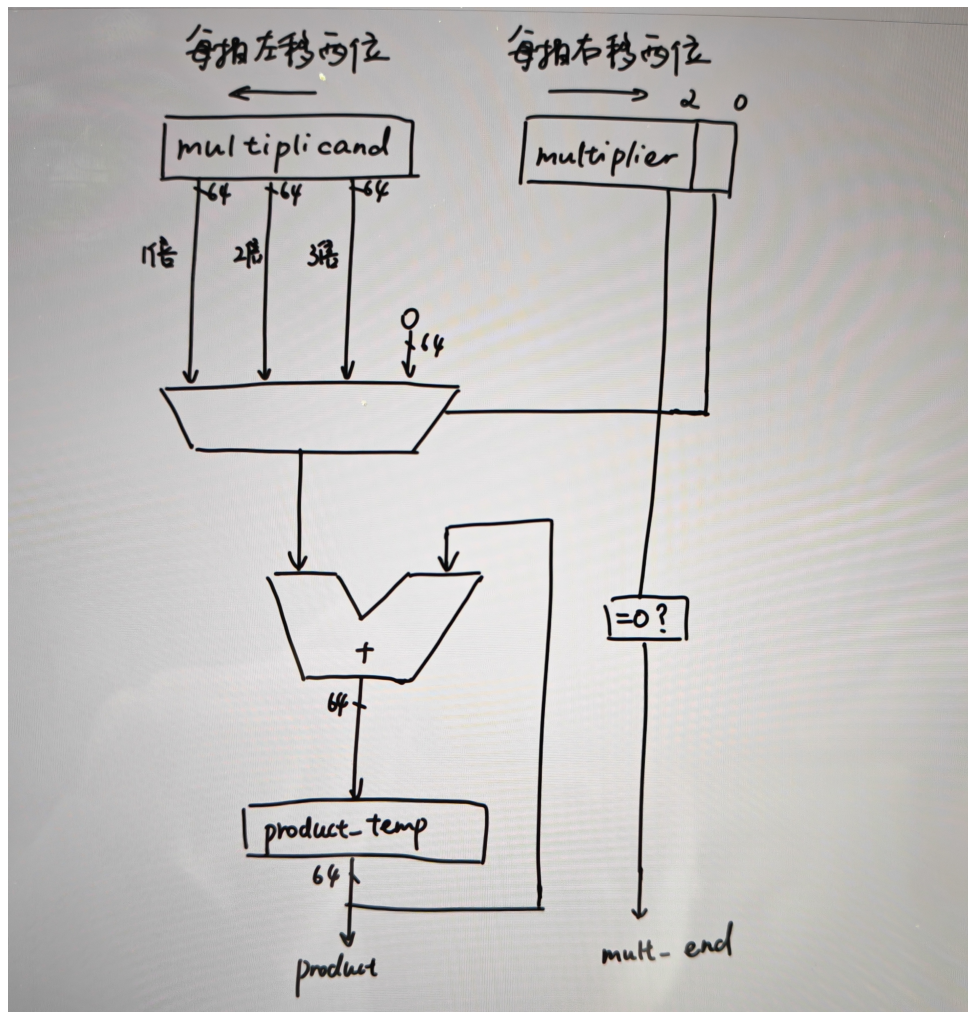


图 1: 2 位并行乘法器硬件结构图

2.2 原理说明

1. 符号与绝对值处理单元：提取操作数符号位，对负数进行取反加 1 得到绝对值。
2. 被乘数寄存器：64 位，每次左移 2 位。
3. 乘数寄存器：32 位，每次右移 2 位。
4. 2 位编码与部分积生成单元：根据乘数低两位选择部分积（0、 $\times 1$ 、 $\times 2$ 、 $\times 3$ ）。
5. 累加寄存器：对部分积进行累加。
6. 控制单元：判断乘数是否为 0，产生乘法结束信号。
7. 符号修正单元：根据符号位异或结果，对最终乘积进行符号修正。

3 实验过程与代码实现

本次实验仅修改 multiply.v 文件，保持其他模块不变。

3.1 核心优化思路

- 原始乘法：每次处理 1 位，需要 32 个时钟周期。
- 优化乘法：每次处理 2 位，只需要 16 个时钟周期。
- 部分积直接由移位和加法生成，硬件最简单、最容易实现。

3.2 优化后 Verilog 代码

```
1  `timescale 1ns / 1ps
2
3  module multiply(           // 乘法器
4      input      clk,       // 时钟
5      input      mult_begin, // 乘法开始信号
6      input  [31:0] mult_op1, // 乘法源操作数1
7      input  [31:0] mult_op2, // 乘法源操作数2
8      output [63:0] product,  // 乘积
9      output      mult_end   // 乘法结束信号
10 );
11
12 //乘法正在运算信号和结束信号
13 reg mult_valid;
14 assign mult_end = mult_valid & ~(|multiplier); //乘法结束信号：乘
    数全0
15 always @(posedge clk)
16 begin
17     if (!mult_begin || mult_end)
18     begin
19         mult_valid <= 1'b0;
20     end
21     else
22     begin
23         mult_valid <= 1'b1;
24     end
25 end
26
```

```

27 //两个源操作取绝对值
28 wire      op1_sign;      //操作数1的符号位
29 wire      op2_sign;      //操作数2的符号位
30 wire [31:0] op1_absolute; //操作数1的绝对值
31 wire [31:0] op2_absolute; //操作数2的绝对值
32 assign op1_sign = mult_op1[31];
33 assign op2_sign = mult_op2[31];
34 assign op1_absolute = op1_sign ? (~mult_op1+1) : mult_op1;
35 assign op2_absolute = op2_sign ? (~mult_op2+1) : mult_op2;
36
37 //被乘数，每次左移2位
38 reg [63:0] multiplicand;
39 always @ (posedge clk)
40 begin
41     if (mult_valid)
42     begin
43         multiplicand <= {multiplicand[61:0],2'b0};
44     end
45     else if (mult_begin)
46     begin
47         multiplicand <= {32'd0,op1_absolute};
48     end
49 end
50
51 //乘数，每次右移2位
52 reg [31:0] multiplier;
53 always @ (posedge clk)
54 begin
55     if (mult_valid)
56     begin
57         multiplier <= {2'b0,multiplier[31:2]};
58     end
59     else if (mult_begin)
60     begin
61         multiplier <= op2_absolute;
62     end
63 end
64
65 // 2位并行部分积生成
66 wire [63:0] partial_product;

```



```

67     assign partial_product = (multiplier[1:0] == 2'b00) ? 64'd0 :
68                               (multiplier[1:0] == 2'b01) ? multiplicand
69                               :
70                               (multiplier[1:0] == 2'b10) ? (
71                                   multiplicand << 1) :
72                                   (multiplicand + (multiplicand << 1));
73
74     //累加器
75     reg [63:0] product_temp;
76     always @ (posedge clk)
77     begin
78         if (mult_valid)
79         begin
80             product_temp <= product_temp + partial_product;
81         end
82         else if (mult_begin)
83         begin
84             product_temp <= 64'd0;
85         end
86     end
87
88     //结果符号处理
89     reg product_sign;
90     always @ (posedge clk)
91     begin
92         if (mult_valid)
93         begin
94             product_sign <= op1_sign ^ op2_sign;
95         end
96     end
97
98     assign product = product_sign ? (~product_temp+1) : product_temp;
99 endmodule

```

4 实验结果与分析

4.1 仿真波形图

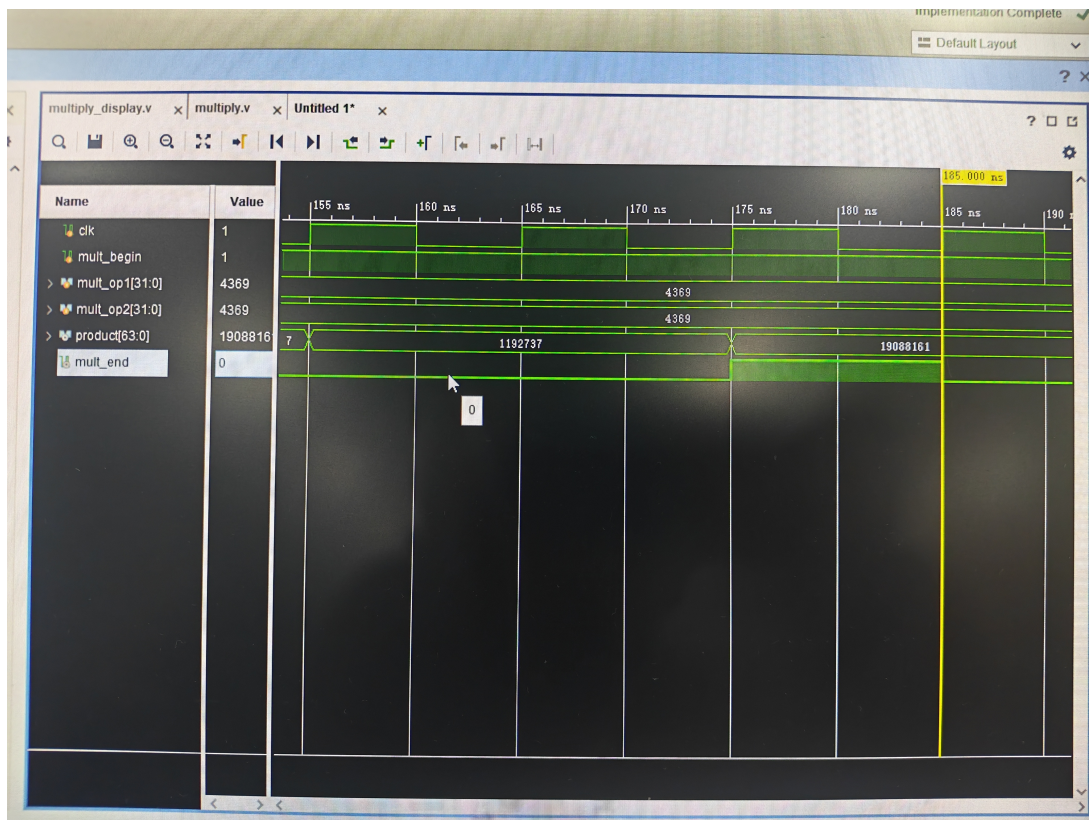


图 2: 乘法器仿真波形图

波形说明:

- 乘法启动信号 `mult_begin` 有效后，乘法器开始工作。
- 乘数每次右移 2 位，被乘数每次左移 2 位，周期数大幅减少。
- 16 个时钟周期后 `mult_end` 拉高，表示乘法完成。
- 输出乘积 `product` 与理论计算值完全一致，结果正确。

4.2 优化前后时钟周期对比

乘法器类型	完成 32 位乘法所需时钟周期
原始串行乘法器	32 周期
2 位并行优化乘法器	16 周期

表 1: 性能对比表

结论：优化后乘法器时钟周期减少 50%，性能提升一倍，硬件实现简单，资源开销小。

4.3 3. 上板验证结果

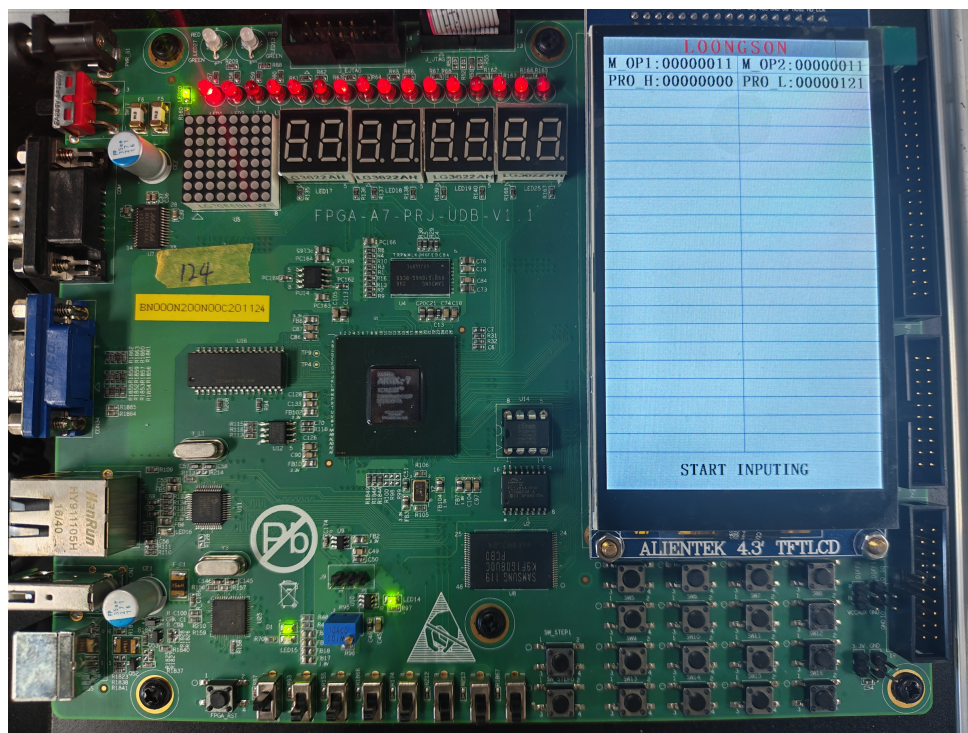


图 3: 实验箱上板验证结果 1

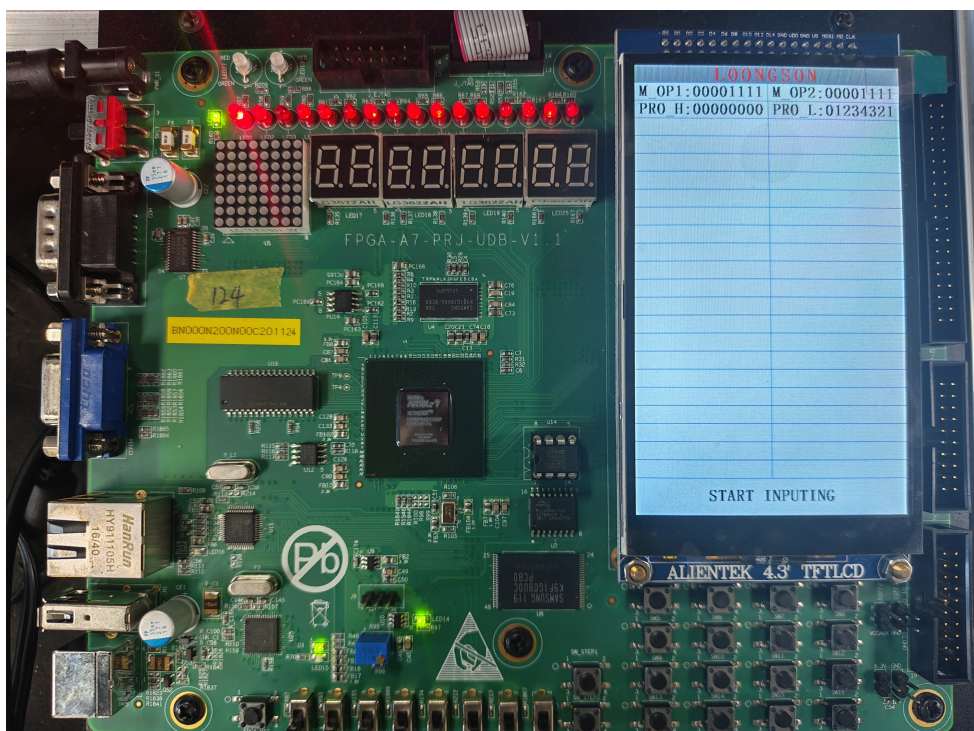


图 4: 实验箱上板验证结果 2

结果说明:

多次测试均正确，优化后乘法器功能完全正常。

5 实验收获与感想

通过本次定点乘法器优化实验，我收获如下：

1. 理解了原始串行乘法器需要逐位计算，周期长、效率低。
2. 掌握了最简单有效的 2 位并行乘法优化方法，只需修改移位位数和部分积判断，即可将周期减半。
3. 学会了部分积的生成方式：使用移位和加法实现 $\times 1$ 、 $\times 2$ 、 $\times 3$ ，无需乘法器，硬件更简洁高效。
4. 完成了从代码编写、功能仿真到上板验证的完整流程，提升了硬件设计能力。
5. 认识到计算机组成原理中“并行化”是提升运算速度的核心思想，简单优化即可带来明显性能提升。

6 实验总结

本次实验成功完成了 32 位定点乘法器的性能优化，采用 2 位并行乘法方式，将运算周期从 32 个时钟周期减少至 16 个时钟周期，性能提升一倍。优化后的乘法器逻辑简单、易于实现、资源占用少，仿真与上板验证均证明结果正确。